

```
1]: import pandas as pd
```

```
2]: data = pd.read_excel('data2.xlsx')
```

```
3]: data.head()
```

```
4]: data.info()
```

```
data.describe()
```

▼ Now Explore the data by visualizing the of values some column of the datasets

we will use some Libraries Matplotlib, Seaborn, and Plotly for Visualization

```
[6]: pip install plotly matplotlib seaborn
```

```
Requirement already satisfied: plotly in c:\users\hp\anaconda3\lib\site-packages (5.24.1)
Requirement already satisfied: matplotlib in c:\users\hp\anaconda3\lib\site-packages (3.10.0)
Requirement already satisfied: seaborn in c:\users\hp\anaconda3\lib\site-packages (0.13.2)
Requirement already satisfied: tenacity>=6.2.0 in c:\users\hp\anaconda3\lib\site-packages (from plotly) (9.0.0)
Requirement already satisfied: packaging in c:\users\hp\anaconda3\lib\site-packages (from plotly) (24.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: numpy>=1.23 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib) (2.1.3)
Requirement already satisfied: pillow>=8 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\hp\anaconda3\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pandas>=1.2 in c:\users\hp\anaconda3\lib\site-packages (from seaborn) (2.2.3)
Requirement already satisfied: pytz>=2020.1 in c:\users\hp\anaconda3\lib\site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\hp\anaconda3\lib\site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\hp\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[7]: import plotly.express as px
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
[8]: fig = px.histogram(data, x='Age', marginal='box', nbins=47, title='Age Distribution')

fig.show()
```

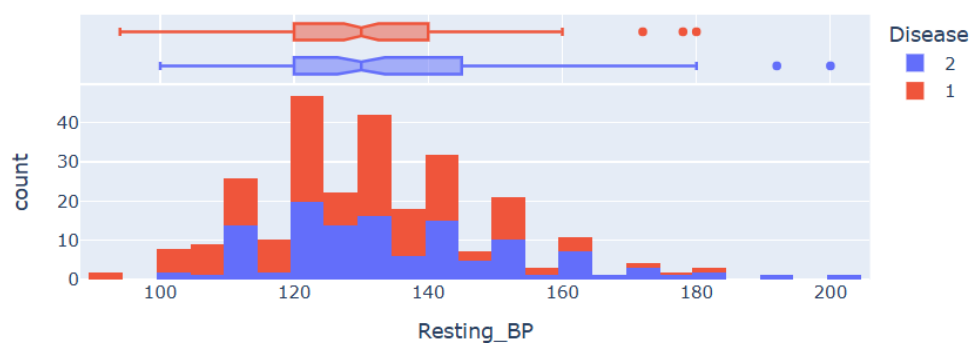
```
[9]: fig = px.histogram(data, x='gender', marginal='box',nbins=47, title='Age Distribution')
fig.show()
```

```
[10]: fig = px.histogram(data, x='chest_pain_type', marginal='box',nbins=47, title='Age Distribution')
fig.show()
```

```
[11]: fig = px.histogram(data, x='Resting_BP', marginal='box',nbins=47, title='Bp Distribution')
fig.show()
```

```
[12]: fig = px.histogram(data, x='Resting_BP', marginal='box', color='Disease', title='Bp Distribution')
fig.show()
```

Bp Distribution



```
[13]: fig = px.histogram(data, x='gender', marginal='box', color='Disease')
fig.show()
```

```
[14]: fig = px.scatter(data, x='Age', opacity=0.8, y='Resting_BP',color='Disease', title='Age with disease')
fig.show()
```

```
[15]: fig = px.violin(data, x='gender', y='Disease')
fig.show()
```

```
[16]: data.corr()
```

```
[17]: #sns.heatmap(data.corr(), cmap="Reds", annot=True)

sns.heatmap(data[['Age', 'gender', 'chest_pain_type', 'Resting_BP', 'Cholestrol', 'Fasting_sugar', 'Resting_ECG', 'Max_heart_rate', 'Exercise_induced_ang',
                  'Slope_peak_excercise', 'No_of_major_vessels', 'Thal', 'Disease']].corr(), annot=True, cmap='coolwarm')
plt.figure(figsize=(14, 7))
plt.show()
```

Linear Regression

```
[20]: import matplotlib.pyplot as plt

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

plt.scatter(x, y)
plt.show()
```

```
[21]: import matplotlib.pyplot as plt
from scipy import stats

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(x):
    return slope * x + intercept

mymodel = list(map(myfunc, x))

plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```

R for Relationship

It is important to know how the relationship between the values of the x-axis and the values of the y-axis is, if there are no relationship the linear regression can not be used to predict anything.

This relationship - the coefficient of correlation - is called r.

The r value ranges from -1 to 1, where 0 means no relationship, and 1 (and -1) means 100% related.

```
[22]: from scipy import stats

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

slope, intercept, r, p, std_err = stats.linregress(x, y)

print(r)

-0.758591524376155
```

The result -0.76 shows that there is a relationship, not perfect, but it indicates that we could use linear regression in future predictions.

▼ Predict Future Values

Predict the speed of a 10 years old car:

```
[23]: from scipy import stats

x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]

slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(x):
    return slope * x + intercept

speed = myfunc(10)

print(speed)
```

▼ Let us create an example where linear regression would not be the best method to predict future values.

```
[24]: import matplotlib.pyplot as plt
from scipy import stats

x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]

slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(x):
    return slope * x + intercept

mymodel = list(map(myfunc, x))

plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```

```
[25]: import numpy
      from scipy import stats

      x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
      y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]

      slope, intercept, r, p, std_err = stats.linregress(x, y)

      print(r)

0.01331814154297491
```

The result: 0.013 indicates a very bad relationship, and tells us that this data set is not suitable for linear regression.

Polynomial Regression

```
[26]: import matplotlib.pyplot as plt

      x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
      y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

      plt.scatter(x, y)
      plt.show()
```

```
[27]: import numpy
      import matplotlib.pyplot as plt

      x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
      y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

      mymodel = numpy.poly1d(numpy.polyfit(x, y, 1))

      myline = numpy.linspace(1, 22, 100)

      plt.scatter(x, y)
      plt.plot(myline, mymodel(myline))
      plt.show()
```

```
[28]: import numpy
      import matplotlib.pyplot as plt

      x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
      y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

      mymodel = numpy.poly1d(numpy.polyfit(x, y, 2))

      myline = numpy.linspace(1, 22, 100)

      plt.scatter(x, y)
      plt.plot(myline, mymodel(myline))
      plt.show()
```

```
[29]: import numpy
import matplotlib.pyplot as plt

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

myline = numpy.linspace(1, 22, 100)

plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()
```

```
[30]: import numpy
from sklearn.metrics import r2_score

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

print(r2_score(y, mymodel(x)))

0.9432150416451027
```

The result 0.94 shows that there is a very good relationship, and we can use polynomial regression in future predictions.

Predict the speed of a car passing at 17:00:

▼ Predict the speed of a car passing at 17:00:

```
[31]: import numpy
from sklearn.metrics import r2_score

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

speed = mymodel(17)
print(speed)
```

```
[32]: import numpy
import matplotlib.pyplot as plt

x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

myline = numpy.linspace(2, 95, 100)

plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()
```

```
[33]: import numpy
from sklearn.metrics import r2_score

x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

print(r2_score(y, mymodel(x)))

0.009952707566680652
```

The result: 0.00995 indicates a very bad relationship, and tells us that this data set is not suitable for polynomial regression.

- ▼ The result: 0.00995 indicates a very bad relationship, and tells us that this data set is not suitable for polynomial regression.

```
[34]: import pandas
from sklearn import linear_model

X = data[['Age', 'gender', 'chest_pain_type', 'Resting_BP', 'Cholestrol', 'Fasting_sugar', 'Resting_ECG', 'Max_heart_rate', 'Exercise_induced_angina', 'Thal']]
y = data['Disease']

regr = linear_model.LinearRegression()
regr.fit(X, y)

print(regr.coef_)
```